

## **Resumen 2do Parcial Programación III 2023.**

### **Colecciones.**

Collection. Es una interfaz, no una clase. Por eso, no se puede construir, sino que hay clases que la implementan.

Provee métodos:

- add(),
- addAll(),
- int size(),
- boolean contains(),
- void clear(),
- boolean remove(),
- boolean addAll(Collection, T elements),
- void copy (destino, origen),
- int frequency,
- Object max(Collection c)
- Object min(Collection c)
- void reverse(List lista)

### **Listas.**

Cada elemento tiene una posición determinada, que se indica con un índice. Las listas vienen definidas en la interfaz List, que ofrece los siguientes métodos:

- void add(int indice, Object o)
- Object get (int indice)
- int indexOf(Object o)
- Object remove (int indice)
- Object set (int indice, Object obj)

Implementaciones: ArrayList, Vector, LinkedList

### **ArrayList.**

Implementa una lista de elementos mediante un array de tamaño variable. Conforme se van añadiendo elementos, el tamaño del array irá creciendo. Tiene una capacidad inicial.

Su implementación no está sincronizada : Si varios hilos acceden al mismo arrayList a la vez puede llegar a haber consistencia en los datos. No es 100% seguro.

### **Vector.**

De implementación similar al ArrayList, con el diferencial de que el Vector si está sincronizado. Es un caso especial. Existe desde las primeras versiones de Java.

Métodos:

- void addElement(Object o)
- Object elementAt(int index)
- void insertElementAt(Object o, int indice)
- boolean removeElement(Object o )

- boolean removeElementAt(int indice)
- void setElement(Object o , int indice)
- int size()

Sobre el Vector se construye el tipo Pila (Stack).

### **LinkedList.**

Se implementa como una lista doblemente enlazada. El coste temporal de este tipo de colecciones es más alto en los casos en los que haya que buscar, colocar o borrar un elemento determinado dado que la tiene que recorrer de principio a fin siempre.

El coste es lineal  $O(n)$ , siendo  $n$  el tamaño de la lista.

Métodos:

- void addFirst(Object o )
- void addLast(Object o )
- Object getFirst()
- Object getLast()
- Object removeFirst()
- Object removeLast()

### **Sets.**

Son grupos de elementos en los que no encontramos ninguno repetido. Consideramos que un elemento está repetido a partir del método equals. Un set podrá tener a lo sumo un elemento null.

La interfaz Set, al igual que List, proporciona métodos para:

- añadir elementos
- eliminar
- obtener un elemento
- localizar un elemento
- iterar sobre la coleccion

### *Iterator.*

Método que recorre los elementos de un set.

```
Iterator it = hs.iterator();
while (it.hasNext())
{
    sout(it.next());
}
```

Tipos de sets: HashSet, LinkedHashSet, TreeSet.

### **HashSet.**

Los objetos se almacenan en una tabla de dispersión (hash). El costo de las operaciones básicas se realizan en tiempo constante  $O(1)$ . Iterar a través de sus elemento es más costoso, ya que recorre todas las entradas de la tabla de dispersión, lo que hace que el coste aumente de acuerdo a la cantidad de elementos insertados así como también la cantidad de entradas de la tabla. El orden de iteración puede diferir del orden de inserción.

### **LinkedHashSet.**

Parecida a la anterior, pero doblemente enlazada. Las operaciones tendrán costo constante, con la carga que supone gestionar los enlaces. Recorrer el conjunto será un poco mejor que en el HashSet, dado que no se recorren todas las entradas de la lista hash, sino tantos elementos como haya insertados. El orden de iteración será el mismo orden en el que fueron insertados.

### **TreeSet.**

Usa una estructura de árbol para el almacenamiento de objetos. El costo de las operaciones básicas será logarítmico con el número de elementos del conjunto  $O(\log n)$

Métodos:

- `Object higher(Object o)` : Devuelve el menor elemento de la colección pero que sea mayor que el elemento pasado por parámetro. Si no existe, devuelve null.
- `Object lower (Object o)` : Devuelve el mayor elemento de la colección pero que sea menor que el elemento pasado por parámetro. Si no existe, devuelve null.
- `Object pollFirst ()` : elimina el primer elemento de la colección. Si está vacía devuelve null.
- `Object pollLast()` : elimina el último elemento de la colección. Si Está vacía devuelve null.

### **Maps.**

No son estrictamente una colección ya que no heredan de la interfaz Collection, lo hacen desde la interfaz Map.

Un mapa es un objeto que relaciona una clave (key) con un valor. Cada mapa contiene un conjunto de claves y a cada clave se le asignará un determinado valor. Tanto la clave como el valor puede ser cualquier Objeto.

Métodos:

- `Object get (Object key)`
- `Object put (Object key, Object value)`
- `Object remove (Object key)`
- `Set keySet()`
- `int size()`

Cómo se recorre un mapa:

```
Iterator it = hm.entrySet().iterator();
```

```
while(it.hasNext)  
{  
    Map.Entry me= (Map.Entry)it.next();  
    sout(me.getKey() + "--" + me.getValues());  
}
```

Implementaciones : HashMap, TreeMap, Hashtable.

**HashMap.**

Utiliza una tabla de dispersión para almacenar la información. Las operaciones básicas se harán en tiempo constante. El costo de iteración dependerá de la cantidad de elementos en la tabla y el número de elementos en el mapa. No se garantiza que se respete el orden de las claves.

**HashTable.**

Parecido al HashMap, pero sincronizada. No se permiten claves nulas.

**TreeMap.**

Utiliza un árbol simétrico para el almacenamiento. El coste de las operaciones básicas es logarítmico. Los elementos se encuentran ordenados de forma ascendente de clave.

Métodos:

- Object ceilingKey()
- Object firstKey()
- Object lastKey()
- Object lowerKey(Object key)
- Object higherKey(Object key)

**Big O.**

Es la comparación entre algoritmos reducida a una sola variable.

*Complejidad Constante  $O(1)$* 

Significa que la cantidad de datos es irrelevante para el algoritmo, siempre va a tardar el mismo tiempo en terminar. Es difícil que un algoritmo no trivial caiga en esta categoría.

*Complejidad Logarítmica  $O(\log n)$* 

El tiempo incrementa conforme aumenta el número de elementos pero de manera logarítmica, por lo que para grandes cantidades de elementos el tiempo es relativamente menor.

*Complejidad lineal  $O(n)$* 

El tiempo que se tarda incrementa linealmente acorde al aumento de cantidad de elementos.

*Complejidad  $O(n \log n)$* 

Combinación de las dos anteriores, hay muchos algoritmos de ordenamiento que funcionan así.

*Complejidad cuadrática  $O(n^2)$* 

Se dispara el tiempo tardado conforme se van agregando elementos. No es eficiente para colecciones con gran cantidad de datos.

## **Genericidad.**

El uso de Object como referencia genérica a la hora de crear una colección que reciba distintas clases de objetos, dado que los errores que puede llegar a generar los vamos a descubrir recién en tiempo de ejecución.

El término *genericidad* se refiere a una serie de técnicas que permiten escribir algoritmos o definir contenedores de forma que puedan aplicarse a un amplio rango de tipo de datos.

- Se hace abstracción del tipo de dato que se le va a proporcionar
- Se parametriza el o los tipos de datos que intervienen.

La genericidad es importante en un lenguaje de programación orientado a objetos, que si bien no es exclusiva de este tipo de lenguajes, en ellos se convirtió en una herramienta muy importante.

Es una propiedad que permite definir una clase o un método sin especificar el tipo de dato; y de esta forma se puede cambiar de clase para adaptarla a distintos usos sin tener que reescribirla; es beneficiosa porque permite escribir un código más seguro y fácil de leer.

### *Por qué?*

Porque la mayoría de los algoritmos que usamos para resolver problemas no dependen del tipo de dato que se procesa.

Programar con genericidad significa escribir un código que se pueda utilizar con distintos tipos de objetos.

### *Casos de uso.*

- Operaciones aplicables a cualquier dato.
- Contenedores cuyas operaciones no dependan del tipo de dato.
- Acceso uniforme a contenedores secuenciales (arrays)
- Algoritmos y contenedores aplicables a tipos de datos para los que esté definida cierta operación.

### **Clase genérica.**

Es una clase parametrizable, con una o más variables tipo.

Sintaxis:

```
class NombreClase <nombre_tipo> {}
```

donde NombreClase es el nombre de la clase genérica y nombre\_tipo el tipo de parámetro genérico; cuya genericidad podemos limitar.

**Restricciones.**

No se puede instanciar una clase genérica con tipos de datos primitivos. Siempre hay que usar clases envoltorio. Tampoco se pueden usar objetos ni arreglos de tipo genérico.

**Convención.**

- E: elemento de una colección.
- K : clave
- N: número
- T: tipo
- V: valor

**Métodos genéricos.**

Un método genérico o plantilla se usa para definir un grupo de métodos que se pueden utilizar con tipos diferentes. Especifica un conjunto indeterminado de métodos con el mismo nombre que pueden invocarse con argumentos de distintos tipos.

Un método genérico puede definirse dentro de una clase ordinaria o bien dentro de una clase genérica.

**Límites al tipo genérico.**

```
public static <T extends Comparable>
```

en este caso T está limitado a las clases que implementen la interfaz comparable.

Las limitaciones pueden ser múltiples:

```
public class <T extends Comparable & I_InterfazQueHaceAlgo> NombreDeClase
```

En la limitación múltiple, los tipos pueden ser:

- Sólo una clase, la cual debe ir al principio
- Todas las interfaces que se deseen, separadas por el ampersand.

**Herencia y genericidad.**

Una clase puede heredar de una clase genérica. La nueva clase puede mantener la genericidad de la clase padre, restringir la genericidad o no ser genérica y especificar un tipo concreto.

## Excepciones.

Son condiciones de errores súbitos que suelen terminar el programa en ejecución.

El manejo de dichas excepciones es el mecanismo previsto por Java para el tratamiento de equivocaciones por parte del usuario.

Las excepciones se lanzan y se capturan (try-catch)

Si al ocurrir un error está activo una porción de código denominado manejador de excepción, el flujo de control se transfiere a la excepción, sino, el flujo se propaga al método que invoca, y mientras no se maneje va ascendiendo entre las clases hasta llegar al main. Si en el main no es manejada, se termina la ejecución.

*El manejo de errores usando excepciones no los evita, sino que permite su detección y su posible reparación.*

Mecanismo:

- try: es un bloque para detectar errores.
- catch: es un manejador para capturar las excepciones del bloque try
- throw: es la expresión que se usa para levantar excepciones
- throws: indica las excepciones que puede elevar un método.
- finally: en un bloque opcional situado después de los catch. Se utiliza para que el programa haga alguna cosa una vez que se realizó todo el mecanismo.

### Jerarquía de excepciones en Java.

Todas las excepciones heredan de la clase Exception. Las RuntimeException son las excepciones *no comprobadas*, es decir que no necesitan ser especificadas en la firma del método. No son errores que dependan de la programación.

Las *excepciones comprobadas*, son aquellas que no están contempladas en la ejecución del programa y por eso deben especificarse en la firma de los métodos que puedan llegar a lanzarse. De esta manera, el IDE sabrá que dentro de dicho método está la posibilidad de que se genere una excepción y puede eventualmente indicar a quien la invoque que dicha excepción necesita ser manejada (a través del correspondiente bloque try-catch)

### Clases de excepciones.

Nosotros podemos definir excepciones para tener un control específico sobre las mismas (cuando se generan, que tipo de datos devuelven y como se desea que sean manejadas). Podemos definir que nuestras clases de excepción hereden de la clase Exception, o bien de otro tipo de clases (Si hereda de RuntimeException no será una excepción comprobada entonces no requerirá de un bloque try-catch)

Las excepciones comprobadas están pensadas para aquellos casos en los que el cliente debe esperar que una operación falle. El cliente debe comprobar que dicha operación funcionó.

Las excepciones no comprobadas están pensadas para aquellos casos en los que no debería fallar la operación; generalmente indican un error en el programa.

**Efectos de una excepción.**

Cuando se lanza una excepción, la ejecución del método que la disparó termina inmediatamente.

No hace falta usar el return, el lanzamiento de una excepción es una incapacidad de resolver correctamente el método e incluye la imposibilidad de retornar un resultado válido.

**Excepciones comprobadas: los requerimientos.**

- Clausula throws
- Bloque try (en la clase que lo invoque)
- Bloque catch.

**Clausula finally.**

Proporciona sentencias que se deben ejecutar cuando se lanza una excepción desde sentencias. Se ejecuta al finalizar el try o el catch (según si el método lanzó una excepción)

**Definir nuevas clases de excepción.**

Es la inclusión de más información dentro de los objetos excepción para brindar un mejor diagnóstico de error y de recuperación.

Debería incluir información que ayude en el manejo de la excepción, sobre todo en los tipos de excepciones comprobadas.

**Recuperación del error.**

Si el error no pudiera ocurrir, se requerirá de un flujo de control más complejo.

La recuperación del error dependerá habitualmente de un nuevo intento de ejecución.

No se puede garantizar el éxito de recuperación de una excepción.

Debería haber rutas de escape que eviten que el intento de recuperación caiga en un loop eterno.



## **Documentación.**

Es un complemento necesario para la obtención de un buen código, es necesario para comunicar a otros usuarios del código en lenguaje humano que es lo que hace cada cosa en nuestro código.

En Java, se genera documentación a través de Javadoc, que genera documentación en formato HTML.

### *Comentarios.*

Los elementos de una clase que se documentan son la definición de la clase, sus campos, sus retornos, constructores y métodos. Siempre se documentan arrancando con un slash y dos asteriscos (/\*\*) y terminan con un asterisco y una barra (\*/).

*Documentación de una clase.* Hacemos una descripción de lo que representa dicha clase, su autor, sus métodos y que versión es de la clase.

*Documentación de un método.* Especificamos el propósito de la misma, que tipo de parámetros recibe y que retorna. Si arroja una excepción, también deberíamos especificarlo.

## Archivos.

Los archivos tienen como finalidad guardar datos de forma permanente; una vez terminada la aplicación los datos almacenados quedan disponibles para que otra pueda recuperarlos, así como para consulta o modificación. Los archivos son dispositivos externos, en contraposición de los datos que se guardan en arreglos, listas, o árboles que están en memoria interna y, por tanto, desaparecen al acabar la ejecución del programa

El manejo de archivos en Java se hace a través de los Streams (también llamado canal o secuencia), que conduce los datos entre el programa y los dispositivos externos. Es una abstracción que se refiere a una corriente que va desde un origen hacia un destino.

La apertura de un archivo supone establecer esta conexión.

Un archivo es una colección de registros relacionados entre si con aspectos comunes y organizados con un propósito específico. Es una estructura diseñada para contener datos de tal manera que puedan ser accedidos, modificados y borrados. Es una secuencia de bytes que representan los datos almacenados. Java dispone de clases para trabajar las secuencias como tipo de datos básicos y objetos.

Un archivo consta de una ruta de acceso y un nombre. Ambas constituyen el identificador del mismo.

### Clase File.

Los constructores de los stream que esperan un archivo se sobrecargan para recibir un objeto de la clase File que el nombre del archivo, su ruta y demás características inherentes al mismo.

Esta clase tiene métodos para conocer las propiedades del archivo, tales como la última modificación, permisos, tamaño, etc.

Los constructores de la clase File permiten inicializar el objeto con el nombre de un archivo y la ruta donde se encuentra e inicializarlo con otro objeto File como ruta y el nombre del archivo.

```
public File(String nombreCompleto)  
public File(String ruta, String nombre)  
public File(File ruta, String nombre)
```

Métodos de la clase File:

```
public boolean exists()      public long length()  
public boolean canWrite()    public long lastModified()  
public boolean canRead()  
public boolean isFile()  
public boolean isDirectory()
```

```
public boolean isAbsolute()
```

Operaciones que se pueden realizar con el archivo:

```
public String getName()  
public String getPath()  
public String getAbsolutePath()  
public boolean setReadOnly()  
public boolean delete()  
public boolean renameTo(File nuevo)  
public boolean mkdir()  
public String[] list()
```

## **InputStream y OutputStream**

Todo el proceso de entrada y salida en Java se hace a través del stream; en los programas hay que crear objetos Stream.

InputStream y OutputStream son dos clases abstractas y declaran métodos que deben sobrecargarse en sus clases derivadas. La primera es la base de todas las clases definidas para flujos de entrada y la segunda para las de salida.

## **FileInputStream y FileOutputStream**

Son clases que se usan para leer o escribir bytes en un archivos. Los constructores de ambas clases permiten crear flujos u objetos asociados a un archivo que se encuentra en un directorio.

Ejemplo:

```
FileOutputStream mf = new FileOutputStream("Temperatura.dat");
```

Métodos de FileInputStream

```
FileInputStream(String nombre) throws FileNotFoundException;  
FileInputStream(File nombre) throws FileNotFoundException;  
int read() throws IOException  
int read(byte[] s) throws IOException  
void close()throws IOException;
```

## Métodos de File OutPutStream

```
FileOutputStream(String nombre) throws IOException;  
FileOutputStream(String nombre, boolean append) throws IOException;  
FileOutputStream(File nombre) throws IOException;  
void write(byte a) throws IOException;  
void write(byte[] s) throws IOException;  
void close() throws IOException;
```

## DataInputStream y DataOutputStream.

Resulta poco práctico trabajar de manera directa con flujos de bytes, los datos que se escriben o se leen en los archivos son más elaborados, de mayor nivel, como enteros, reales, cadenas de caracteres, etcétera.

Estas clases filtran secuencias de bytes y los organizan para formar datos de tipo primitivo; así se pueden escribir o leer directamente datos de tipo char, byte, long, double, etc.

*DataInputStream*. declara el comportamiento de los flujos de entrada, con los métodos de entrada de cualquier tipo básico: *readInt()*, *readDouble()*, *readUTF()*. Deben asociarse a un flujo de bytes de tipo *InputStream*, generalmente *FileInputStream*.

```
FileInputStream gs = new FileInputStream("nube.dat");  
DataInputStream ent = new DataInputStream(gs);
```

## Métodos de Data InputStream:

- *boolean readBoolean()*
- *byte readByte()*
- *short readShort()*
- *char readChar()*
- *int readInt()*
- *long readLong()*
- *float readFloat()*
- *double readDouble()*
- *String readUTF()*

*DataOutputStream*. La aplicación que vaya a leer un archivo creado con los métodos de un flujo de este tipo debe tener en cuenta que los de lectura deben corresponderse con los tipos que fueron escritos. Así que si se hizo un *writeInt()* ahora deberá hacerse un *readInt()*, y así con cada tipo de dato que se quiera leer.

Métodos de `DataOutputStream`:

- `writeBoolean(boolean v)`
- `writeInt(int v)`
- `writeLong(long v)`
- `writeFloat(float v)`
- `writeDouble(double v)`
- `writeUTF(String cad)`

### Archivos de objetos.

Para que un objeto persista una vez que termina la ejecución de una aplicación se debe guardar en un archivos de objetos; por cada objeto que se almacena en un archivo se graban características de la clase y atributos del objeto, las clases `ObjectInputStream` y `ObjectOutputStream` se diseñan para crear flujos de entrada y salida de objetos persistentes.

Para crear dichos objetos persistentes debería implementarse la interfaz `Serializable`, que simplemente indica que las instancias de esta clase podrían llegar a grabarse en un archivo.

Ejemplo:

```
public class Racional implements Serializable{...}
```

Los atributos de una clase serializable que se declaren *transient* no se escriben al persistir los datos.

**ObjectOutputStream.** Los flujos de esta clase se utilizan para grabar objetos persistentes. El método `writeObject()` escribe cualquier objeto de una clase serializable en el flujo de bytes asociado, puede elevar excepciones de tipo `IOException`.

Ejemplo :

```
FileOutputStream bn = new FileOutputStream("datosRac.dat");
ObjectOutputStream fobj = new ObjectOutputStream(bn);
Racional rd = new Racional(1,7);
fobj.writeObject(rd);
```

**ObjectInputStream.** Los objetos guardados con `ObjectOutputStream` se recuperan y se leen con esta clase, que es una clase que hereda de `InputStream` que implementa la interfaz `DataInput`, por eso dispone de varios métodos de entrada (`read`), siendo el más importante de ellos el `readObject()`, el cual lee un objeto del flujo de entrada, escrito eventualmente a través de `writeObject()`.

Tiene como entrada otro flujo de cualquier tipo derivado de `InputStream`.

Ejemplo:

```
FileInputStream bn = new FileInputStream("datosRac.dat");  
ObjectInputStream fobj = new ObjectInputStream (bn);  
Racional rac = (Racional) fobj .readObject();
```

## JSON.

JSON (Javascript Object Notation) es un formato para intercambio de datos liviano, basado en texto e independiente del lenguaje de programación.

Puede representar dos tipos de datos, objetos y arrays. Un objeto es una colección no ordenada de cero o más pares de nombre/valor. Por otro lado, un array es una secuencia ordenada de cero o más objetos. Los valores pueden ser Strings, números, booleanos, nulos.

Todos los sitios web populares ofrecen JSON como formato para intercambio de servicios. Es gratis, libre, está documentado y su uso es muy aceptado por la comunidad del desarrollo.

*JsonObject*. Sirve para la creación de nuestros objetos JSON. Cuenta con un método `.put(String key, Object value)` sobrecargado para todo tipo de datos. El método requiere de una key y un valor. Este método arroja una excepción de tipo `JSONException`, que es una excepción comprobada.

Cuenta con un método `.get(String key)` para obtener el valor de una clave

*JSONArray*. Sirve para representar arreglos de `JsonObject` o cualquier tipo de dato. Los elementos dentro del arreglo se separan por coma.

Cuenta con un método `.put(Object value)` para ingresar elementos al arreglo. También cuenta con un método `.get(int index)` para acceder a un elemento en particular.

El método `.length()` arroja la longitud del mismo.

### Consideraciones para JSON.

Tanto el `JsonObject` como el `JSONArray` tienen un constructor donde podemos enviar el texto que representa el árbol JSON. Automáticamente se crean los objetos y los arreglos en memoria.

Siempre debemos conocer la estructura de nuestro árbol.